

3-zu-8-Decoder

Projektbericht

Deniz Ada Abali
B.Sc.-Student Elektrotechnik | Universität Innsbruck & UMIT TIROL
GitHub: github.com/Eng-DenizAbali
Datum: September 2026

Zielsetzung

Decoder zählen zu den essenziellen Bausteinen der Digitaltechnik. Ihre zentrale Bedeutung liegt darin, dass sie in Abhängigkeit von den anliegenden Eingangssignalen einen bestimmten Ausgangswert bzw. Ausgangskanal gezielt ansteuern und aktivieren. Das Ziel dieses Versuchs ist der Entwurf, die VHDL-basierte Implementierung und die messtechnische Überprüfung eines 3-zu-8 Decoders auf dem FPGA-Board TerasIC DE0-Nano.

Die Synthese und das Pin-Assignment erfolgen in Intel Quartus Prime. Abschließend soll die Funktionsfähigkeit sowohl durch eine Verhaltenssimulation als auch durch manuelle Tests über die On-Board-Schalter und LEDs validiert werden.

1 Vorbereitung

Der erste Schritt dieses Versuchs besteht darin, die Bitbreite der Ein- und Ausgangssignale festzulegen, da die Dekodierlogik in Abhängigkeit von der Eingangskombination gezielt einen der Ausgänge aktiviert. Für diesen Entwurf wurde eine Eingangsbreite von 3 Bit gewählt. Mit den resultierenden $2^3 = 8$ möglichen Eingangskombinationen (000_2 bis 111_2 , entsprechend 0 bis 7 dezimal) können somit 8 separate Ausgangsleitungen gezielt angesteuert werden (3-zu-8-Decoder).

Wie aus der Wahrheitstabelle (Tabelle 1.1) hervorgeht, wird der Eingangskombination '000' das Ausgangssignal Y_0 zugeordnet, '001' entspricht Y_1 usw. Die Ursache für diese Zuordnung liegt darin, dass die 3-Bit-Eingangswerte im Binärsystem den Dezimalzahlen von 0 bis 7 entsprechen und somit schrittweise die jeweiligen Ausgangsleitungen aktivieren.

Tabelle 1. Wahrheitstabelle des 3-zu-8-Decoders

Dezimal	Eingänge			Ausgänge							
Dez	A_2	A_1	A_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0	0	0	1	0
2	0	1	0	0	0	0	0	0	1	0	0
3	0	1	1	0	0	0	0	1	0	0	0
4	1	0	0	0	0	0	1	0	0	0	0
5	1	0	1	0	0	1	0	0	0	0	0
6	1	1	0	0	1	0	0	0	0	0	0
7	1	1	1	1	0	0	0	0	0	0	0

In der Wahrheitstabelle sind die Eingangsbits mit A (A_2, A_1, A_0) und die Ausgangsleitungen mit Y ($Y_7 \dots Y_0$) gekennzeichnet. Das funktionale Ziel der Schaltung besteht darin, bei Anliegen einer bestimmten Eingangskombination das in der Tabelle definierte Ausgangsmuster am Ausgang bereitzustellen.

Das entsprechende Logikschaltbild des 3-zu-8-Decoders ist in Abbildung 1 dargestellt.

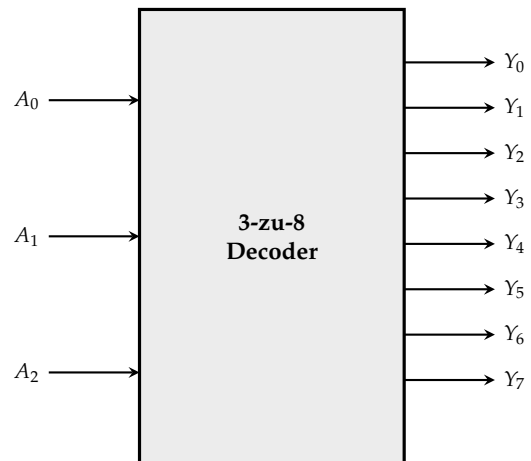


Abbildung 1. Blockschaltbild des 3-zu-8-Decoders

Einer der wichtigsten Teile des Versuchs ist, ein VHDL-Code fehlerfrei zu implementieren, ihn erfolgreich zu kompilieren und das Schaltungsdesign anschließend durch eine Verhaltenssimulation zu verifizieren. Nach erfolgreicher Durchführung und Verifikation dieser Schritte wird die generierte Konfigurationsdatei (Bitstream) auf das FPGA-Board geladen, um das Schaltverhalten der Schaltung auf der realen Hardware experimentell zu überprüfen. Die genannten Entwicklungsschritte – von der Synthese bis zum Pin-Mapping – werden vollständig in der Entwicklungsumgebung Intel Quartus Prime durchgeführt, bevor das Design abschließend auf dem FPGA-Board TerasIC DE0-Nano in Betrieb genommen wird.

Der VHDL-Code gliedert sich in das Schaltungsdesign des Decoders und eine zugehörige Testbench, mit der alle Eingangskombinationen vor der Hardware-Implementierung simuliert werden.

Zunächst wird das Schaltungsdesign des Moduls betrachtet. Hier wird vor dem Architektur-Teil die erforderliche IEEE-Bibliothek mit dem Paket STD_LOGIC_1164 in das Design eingebunden, um standardisierte

Datentypen wie `std_logic` und `std_logic_vector` für die Schnittstellendefinition nutzen zu können.

Listing 1. Entity-Deklaration des 3-zu-8-Decoders

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity decoder3zu8 is
5     port (
6         A : in  std_logic_vector(2 downto 0);
7         Y : out std_logic_vector(7 downto 0)
8     );
9 end decoder3zu8;
```

In Listing 1 ist die Entity-Deklaration des Decoders abgebildet. Der 3-Bit-Eingang *A* ist als Vektor `std_logic_vector(2 downto 0)` und der 8-Bit-Ausgang *Y* als `std_logic_vector(7 downto 0)` definiert, wodurch alle Ein- und Ausgangsleitungen gebündelt als Busse angesteuert werden können.

Im Architektur-Block des Decoders wird eine selektive Signalzuweisung mittels der `with ... select`-Struktur verwendet, um die Eingangskombinationen des 3-Bit-Vektors *A* direkt den entsprechenden Ausgangsmustern von *Y* zuzuordnen.

Listing 2. Architektur des 3-zu-8-Decoders mittels selektiver Signalzuweisung

```
1 architecture Behavioral of decoder3zu8 is
2 begin
3     with A select
4         Y <= "00000001" when "000",
5              "00000010" when "001",
6              "00000100" when "010",
7              "00001000" when "011",
8              "00010000" when "100",
9              "00100000" when "101",
10             "01000000" when "110",
11             "10000000" when "111",
12             "00000000" when others;
13 end Behavioral;
```

Die `when others`-Struktur stellt dabei sicher, dass undefinierte oder unvollständige Eingangszustände (z. B. 'U' oder 'X') deterministisch auf den Ausgangsvektor "00000000" abgebildet werden, um unerwünschte Latch-Strukturen zu vermeiden.

Zur Verifikation des Schaltungsentwurfs wird eine Testbench implementiert. Diese ermöglicht es, die Funktionslogik noch vor dem Laden der Konfigurationsdatei (Bitstream) auf das FPGA-Board durch gezielte Stimuli-Signale auf logische Fehler zu überprüfen. Innerhalb der Testbench werden interne Signale definiert. Diese Signale werden als *tb_A* und *tb_Y* bezeichnet und entsprechen dem Eingang *A* bzw. dem Ausgang *Y*. Da die Testbench als geschlossene Simulationsumgebung dient und keine physischen Schnittstellen nach außen besitzt, bleibt deren Entity leer.

Listing 3. Entity und Signaldeklaration der Testbench

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity tb_decoder3zu8 is
5 end tb_decoder3zu8;
6
7 architecture sim of tb_decoder3zu8 is
8
9     signal tb_a : std_logic_vector(2 downto 0);
10    signal tb_b : std_logic_vector(7 downto 0);
```

Nachfolgend wird das zuvor erstellte Schaltungsdesign über die direkte Instanziierung (`entity work.decoder3zu8`) in die Testbench eingebunden. Mittels der `port map`-Anweisung werden die lokalen Stimuli- und Beobachtungssignale (*tb_a*, *tb_b*) direkt den physikalischen Ein- und Ausgangsports des instanziierten Decoders zugeordnet.

Im Stimuli-Prozess (`stim_proc`) werden nacheinander alle acht möglichen Eingangskombinationen des 3-Bit-Vektors *tb_a* mit einer Haltezeit von jeweils 20 ns durchlaufen. Die abschließende `wait`-Anweisung stoppt den Prozess, um eine zyklische Wiederholung der Testsequenz zu verhindern.

Listing 4. Stimuli-Prozess zur sequentiellen Ansteuerung des Decoders

```
1 stim_proc: process
2 begin
3     tb_a <= "000"; wait for 20 ns;
4     tb_a <= "001"; wait for 20 ns;
5     tb_a <= "010"; wait for 20 ns;
6     tb_a <= "011"; wait for 20 ns;
7     tb_a <= "100"; wait for 20 ns;
8     tb_a <= "101"; wait for 20 ns;
9     tb_a <= "110"; wait for 20 ns;
10    tb_a <= "111"; wait for 20 ns;
11    wait;
12 end process;
13 end sim;
```

Nach Fertigstellung der Testbench werden sowohl das Design als auch die Testbench über die Option „*Compile all*“ kompiliert. Erscheint die Erfolgsmeldung (grüne Statussymbole), war die Kompilierung fehlerfrei und die Simulation kann gestartet werden.

Das in Abbildung 2 dargestellte Zeitdiagramm bestätigt die fehlerfreie Funktion des Schaltungsentwurfs. Das Eingangssignal *tb_a* durchläuft im Abstand von jeweils 20 ns die binären Zustände von "000" bis "111". Das Ausgangssignal *tb_b* zeigt das erwartete One-Hot-Verhalten, bei dem zu jedem Zeitpunkt exakt ein Bit aktiv auf logisch '1' gesetzt wird ("00000001" bis "10000000"). Damit ist die geforderte Funktionalität des 3-zu-8-Decoders vollständig verifiziert.

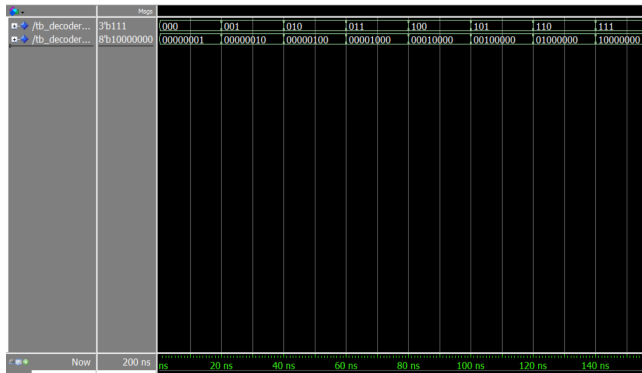


Abbildung 2. Simulationsergebnis des 3-zu-8-Decoders in binärer Darstellung

Nachdem die Funktionalität des Designs erfolgreich verifiziert wurde, kann der Entwurf nach der Synthese und Pin-Zuweisung auf das FPGA-Board geladen werden.

2 Durchführung

Beginn wird die Entwicklungsumgebung Quartus Prime Lite geöffnet, um die Pin-Zuweisung der Ein- und Ausgangssignale im Pin Planner vorzunehmen und die Schaltung nach erfolgreicher Synthese auf das FPGA-Board zu übertragen.

Anschließend wird in den Projekteinstellungen das Ziel-FPGA ausgewählt. Hierbei wird der Baustein EP4CE22F17C6 aus der Cyclone IV E-Familie im 256-Pin FBGA-Gehäuse definiert. Diese Auswahl stellt sicher, dass die spätere Synthese, das Fitter-Routing und die physikalischen I/O-Spannungsstandards exakt auf die Hardware-Spezifikationen des Entwicklungsboards abgestimmt werden.

Vor der Pin-Zuweisung ist es ein notwendiger Schritt, das Design in das Quartus-Prime-Projekt einzubinden, wodurch die Ein- und Ausgangsports des Entwurfs definiert werden. Dies geschieht, indem über das Menü *File* → *New...* eine neue VHDL-Datei angelegt und der Quellcode eingefügt wird (bzw. eine bestehende Datei über *Project* → *Add/Remove Files in Project...* hinzugefügt wird).

Um die syntaktische und logische Korrektheit des Designs sicherzustellen, wird über die Task-Leiste die Funktion *Start Analysis & Synthesis* ausgeführt. Nach erfolgreichem Durchlauf wird der Status durch ein grünes Häkchen bestätigt. Dieser Schritt ist zwingend erforderlich, damit Quartus Prime die definierten Ein- und Ausgangsports (A und Y) der Entity registriert und diese anschließend im *Pin Planner* zur physischen Zuordnung zur Verfügung stehen.

Vor der finalen Übersetzung und Programmierung des FPGA-Bausteins müssen die logischen Ein- und Ausgangsports den tatsächlichen Hardware-Elementen auf dem Board zugeordnet werden. Diese Zuordnung erfolgt im *Pin Planner* von Quartus Prime (siehe Abbildung 3). In Tabelle 2 sind die gewählten Pin-Positionen für die drei Eingänge und acht Ausgänge zusammengefasst.

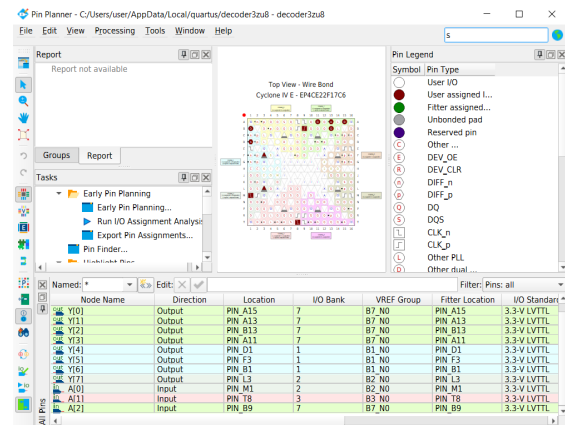


Abbildung 3. Pin-Zuweisung des 3-zu-8-Decoders im Quartus Prime Pin Planner

Hierbei wurden die drei Eingangsports A[2..0] den On-Board-DIP-Schaltern SW[2..0] und die acht Ausgangsports Y[7..0] den grünen Leuchtdioden LED[7..0] des DE0-Nano-Boards zugewiesen, wie in Tabelle 2 aufgeführt ist.

Tabelle 2. Übersicht der Pin-Belegung auf dem FPGA-Entwicklungsboard (EP4CE22F17C6)

Signal	Richtung	Pin-Number	I/O Standard
A[0]	Input	PIN_M1	3.3-V LVTTTL
A[1]	Input	PIN_T8	3.3-V LVTTTL
A[2]	Input	PIN_B9	3.3-V LVTTTL
Y[0]	Output	PIN_A15	3.3-V LVTTTL
Y[1]	Output	PIN_A13	3.3-V LVTTTL
Y[2]	Output	PIN_B13	3.3-V LVTTTL
Y[3]	Output	PIN_A11	3.3-V LVTTTL
Y[4]	Output	PIN_D1	3.3-V LVTTTL
Y[5]	Output	PIN_F3	3.3-V LVTTTL
Y[6]	Output	PIN_B1	3.3-V LVTTTL
Y[7]	Output	PIN_L3	3.3-V LVTTTL

Nach der Durchführung der Pin-Zuweisungen (Pin Assignments) wird das gesamte Design über das Menü *Processing* → *Start Compilation* vollständig synthetisiert, geroutet (Fitter) und assembliert. Durch diesen vollständigen Kompilierungsvorgang wird im Modul *Assembler* die binäre Konfigurationsdatei mit der Endung *.sof* (SRAM Object File) im Projektordner *output_files* generiert.

Sobald alle Einzelschritte mit grünen Häkchen erfolgreich abgeschlossen sind, wird das Werkzeug *Programmer* über das Menü *Tools* geöffnet. In diesem Menü wird die Hardware (USB-Blaster) sowie die zuvor erzeugte *.sof*-Datei ausgewählt. Durch Klicken auf *Start* wird die Bitstream-Datei über die JTAG-Schnittstelle in den flüchtigen SRAM-Speicher des FPGA-Bausteins übertragen. Erreicht der Fortschrittsbalken 100 % (Successful), ist das Design betriebsbereit auf das Board geladen.

Ergebnisse

Abschließend wurde das Design erfolgreich auf das FPGA-Entwicklungsboard TerasIC DE0-Nano übertragen. Die Funktionsfähigkeit der Schaltung wurde durch das manuelle Anlegen aller acht Binärkombinationen (000_2 bis 111_2) über die drei On-Board-DIP-Schalter SW[2..0] messtechnisch überprüft. Wie in Abbildung 4 exemplarisch dargestellt ist, leuchtete bei Anlegen der Eingangskombination 001_2 (SW[2..0] = "001") ausschließlich die Leuchtdiode LED[1].

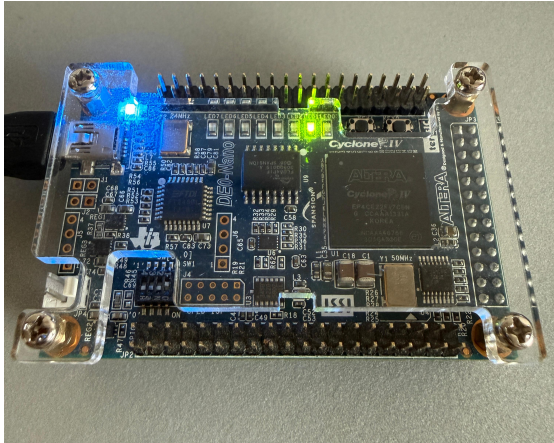


Abbildung 4. Hardware-Validierung des 3-zu-8-Decoders auf dem TerasIC DE0-Nano

Beim Umschalten auf die Kombination 101_2 (SW[2..0] = "101") wurde entsprechend die Diode LED[5] aktiviert, während alle übrigen LEDs dunkel blieben. Auf diese Weise wurden sämtliche in Tabelle 1 definierten Zustände systematisch durchlaufen. Die Ausgangssignale an den acht LEDs LED[7..0] zeigten durchgehend das erwartete One-Hot-Verhalten, womit die korrekte logische Funktionsweise des 3-zu-8-Decoders auf der realen Hardware lückenlos nachgewiesen werden konnte. (Abbildung 5: Hardware-Validierung für die Eingangskombination 101_2)

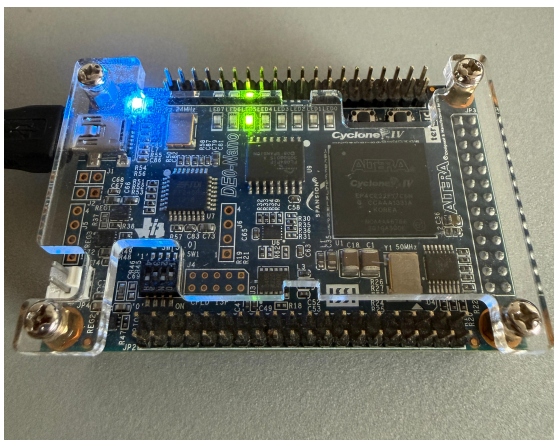


Abbildung 5. Hardware-Validierung des 3-zu-8-Decoders auf dem TerasIC DE0-Nano

Kurzzusammenfassen

- Im Rahmen dieses Versuchs wurde ein 3-zu-8-Decoder erfolgreich in VHDL entworfen sowie alle erforderlichen Definitionen, das Design und die Funktionsweise der Schaltung detailliert beschrieben.
- Nachdem der Schaltungsentwurf abgeschlossen war, wurde das Verhalten des Moduls durch eine Simulation verifiziert.
- Anschließend wurde das Design erfolgreich auf das FPGA-Board übertragen.
- Sowohl die Verhaltenssimulation in ModelSim als auch die hardwarenahe Validierung über die DIP-Schalter und LEDs bestätigten das spezifizierte One-Hot-Verhalten lückenlos.
- Alle acht Eingangskombinationen führten zuverlässig zur Aktivierung des jeweils korrespondierenden Ausgangsbits.

Diskussion

Im vorliegenden Design wurde der Ausgang als zusammenhängender 8-Bit-Vektor (`std_logic_vector(7 downto 0)`) implementiert. Alternativ hätten stattdessen acht separate 1-Bit-Ausgänge (Y_0 bis Y_7) definiert werden können. Dies würde zwar zu einem deutlich längeren und redundanten VHDL-Code führen, böte jedoch den Vorteil, einzelne Ausgangsleitungen im Top-Level-Design ohne vorheriges Aufsplitten (Bit-Slicing) direkt anzusprechen. Die gewählte busorientierte Vektordarstellung sorgt hingegen für eine kompakte, besser skalierbare Schaltungsstruktur und minimiert potenzielle Fehlerquellen bei der Signalzuweisung im Pin Planner.

3 References

- [1] J. Reichardt und B. Schwarz, *VHDL-Synthese: Entwurf digitaler Schaltungen und Systeme*. De Gruyter Studium, 2020.
- [2] Terasic Technologies Inc., *DE0-Nano User Manual*, Rev. 1.8, 2015.
- [3] Intel Corporation, *Intel Quartus Prime Standard Edition User Guide: Design Implementation*, 2021.